

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

Course Name	Computer Vision with OpenCV
Course Code	DSA02
Course Outcome	CO5: Understand video processing – motion computation – 3D vision – geometry (BTL 6)
Name and Reg No	V M Sai Bhargav – 192324126
Questions Answered	Q1, Q2, and Q5

Question 1 (10 Marks): Real-Time Face Recognition System (Edge Deployment)

Problem Understanding & Constraints

A face recognition system must run on a low-power edge device (e.g., Raspberry Pi), meeting four constraints:

- Real-time processing at under 30 ms latency per frame
- Robustness to varying lighting conditions
- Limited memory and CPU resources
- Detection and recognition against a predefined dataset of known faces

Pipeline Design

1. Image Acquisition — Capture live video using `cv2.VideoCapture(0)` at a reduced resolution (320×240) to cut the per-frame data volume and processing cost.

2. Preprocessing — Convert frames to grayscale with `cv2.cvtColor()` to reduce channel count by a factor of three, then apply CLAHE (`cv2.createCLAHE()`) rather than plain histogram equalization, since CLAHE normalizes contrast in local regions and copes with uneven or backlit lighting without over-amplifying noise.

3. Feature Detection — Detect faces using a Haar Cascade or LBP cascade classifier (`cv2.CascadeClassifier`) instead of a deep-learning detector such as MTCNN or a DNN model. Cascades run efficiently on CPU-only edge hardware, directly satisfying the resource constraint.

4. Recognition — Extract features and match identity using the LBPH (Local Binary Pattern Histogram) recognizer (`cv2.face.LBPHFaceRecognizer_create()`). LBPH is preferred over Eigenfaces or Fisherfaces because it is inherently more tolerant to illumination changes and is computationally lightweight.

5. Output — Draw a bounding box and identity label with `cv2.rectangle()` and `cv2.putText()`, then pass the result to any downstream action such as logging or an alert.

Justification

Downscaling combined with grayscale conversion and the Haar+LBPH pipeline keeps memory and CPU usage low, satisfying constraint C, while still completing detection and recognition well within the real-time budget, satisfying constraint A. CLAHE preprocessing directly addresses constraint B (variable lighting), and LBPH matching against the enrolled dataset satisfies constraint D. Each design choice trades a small amount of raw accuracy — compared to deep-learning alternatives — for the efficiency the edge device demands, which is the correct trade-off given the stated constraints.

Question 2 (10 Marks): Automated Video Surveillance with Alert System

Problem Understanding & Constraints

A security firm needs a surveillance system meeting four constraints:

- Detect motion and unusual activity
- Generate real-time alerts
- Work with standard CCTV feeds
- Minimize false positives in dynamic environments

Pipeline Design

- 1. Image Acquisition** — Read the live or recorded CCTV stream using `cv2.VideoCapture()` over an RTSP URL or video file.
- 2. Preprocessing** — Apply Gaussian blur (`cv2.GaussianBlur()`) to each frame to suppress sensor noise before any comparison across frames is performed.
- 3. Background Modeling** — Model the scene background using `cv2.createBackgroundSubtractorMOG2()` with shadow detection enabled, rather than simple frame differencing. MOG2 continuously adapts to gradual background changes such as lighting drift or swaying trees, which directly addresses dynamic environments.
- 4. Motion Analysis** — Clean the resulting foreground mask with morphological opening and closing (`cv2.morphologyEx()`) to remove speckle noise, then extract blobs with `cv2.findContours()` and discard any contour below an area threshold, since small contours generally correspond to noise rather than genuine activity.
- 5. Alert Logic** — Raise an alert only when a qualifying contour persists across several consecutive frames. This temporal filter rejects one-off flickers that would otherwise register as false motion events.
- 6. Output** — Draw bounding boxes and a timestamp overlay with `cv2.rectangle()` and `cv2.putText()`, then dispatch the alert (log entry, notification, or similar).

Justification

MOG2 background subtraction handles gradually changing scenes far better than static frame differencing, directly reducing false triggers from environmental noise. Morphological filtering plus an area threshold and a temporal-persistence check together target constraint D specifically, since each layer removes a different class of spurious detection. Because contour-based analysis is far cheaper than a deep object detector, the system still meets the real-time alerting requirement (constraint B) on ordinary CCTV hardware (constraint

C), while constraint A — motion and unusual-activity detection — is satisfied by the background-subtraction stage itself.

Question 5 (10 Marks): Smart Traffic Monitoring System

Problem Understanding & Constraints

A city authority requires a traffic monitoring system meeting four constraints:

- Detect vehicles from live video streams
- Track movement across frames
- Count vehicles and display statistics
- Operate continuously with minimal delay

Pipeline Design

- 1. Image Acquisition** — Capture the live traffic feed using `cv2.VideoCapture()`.
- 2. Preprocessing** — Convert frames to grayscale and apply Gaussian blur to reduce noise before detection.
- 3. Vehicle Detection** — Apply MOG2 background subtraction and filter the resulting contours by size and aspect ratio to isolate vehicle-shaped blobs. This is computationally cheaper per frame than running a deep-learning detector on every frame, which supports the minimal-delay constraint.
- 4. Tracking** — Track detected vehicles frame-to-frame using a centroid-tracking algorithm: match each new centroid to the nearest existing tracked centroid by Euclidean distance and assign a persistent ID. This is chosen over heavier trackers such as DeepSORT because it keeps latency low while still maintaining identity across frames.
- 5. Counting** — Define a virtual counting line across the road. Increment the vehicle count only when a tracked centroid crosses the line, using the sign of its displacement to determine direction and to avoid counting the same vehicle twice.
- 6. Visualization** — Draw the counting line, bounding boxes, vehicle IDs, and a running count overlay using `cv2.line()`, `cv2.rectangle()`, and `cv2.putText()`.

Justification

Pairing lightweight background subtraction with centroid tracking, instead of a heavier deep-learning detection-and-tracking stack, satisfies the continuous, minimal-delay requirement of constraint D. Frame-to-frame ID persistence fulfills the tracking requirement of constraint B, and the direction-aware line-crossing logic ensures each vehicle is counted exactly once, satisfying constraint C without double-counting. Constraint A is met directly by the detection stage identifying vehicles in each incoming frame.

Declaration: *I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*